You said:

box-selection-syst...
File

Assignment - Pyth...
Document

so i have this assignment and this box-selection-system is the solution tell me step by step what should i do next

Worked for 13s

Analyzed

Analyzed

ChatGPT said:

Yes. I checked **both the assignment and the `box-selection-system.zip` solution**. The good news is that the solution already contains most of the core Django implementation, tests, GitHub Actions, `README.md`, `AI_USAGE.md`, and `TEST_OUTPUT.md`.

The assignment specifically requires a Django box-recommendation system and asks for a GitHub repo, README, AI usage record, chat transcript, learning section, test cases, and test-run output.

Assignment - Python_Django Deve... Assignment - Python_Django Deve...

What you should do next

I recommend **not changing everything at once**. Follow this order:

Step 1 — Extract the ZIP

Extract:

`box-selection-system.zip`

You should get:

```
box-selection-system/
│
├── boxes/
├── config/
├── .github/
├── manage.py
├── requirements.txt
├── README.md
```

```
├── AI_USAGE.md
├── TEST_OUTPUT.md
└── .gitignore
```

The project already has:

- Django models for Product, Box, Order and OrderItem
- Box-selection logic
- REST-style API endpoints
- Unit tests
- Django API tests
- GitHub Actions workflow

So **don't start coding from scratch**.

Step 2 — Open the project in VS Code

Open the extracted:

`box-selection-system`

in VS Code.

Open the terminal inside VS Code.

Step 3 — Create a virtual environment

Run:

```
python -m venv venv
```

Then activate it on Windows:

```
venv\Scripts\activate
```

You should see something like:

```
(venv) PS C:\...\box-selection-system>
```

Step 4 — Install dependencies

Run:

```
pip install -r requirements.txt
```

Then check:

```
python --version
```

and:

```
pip list
```

Step 5 — Check Django

Run:

```
python manage.py check
```

Ideally you should get:

```
System check identified no issues (0 silenced).
```

If you get an error, **stop there and send me the error** rather than changing random files.

Step 6 — Create the database

Run:

```
python manage.py makemigrations
```

Then:

```
python manage.py migrate
```

This will create the SQLite database.

Step 7 — Run the existing tests FIRST

This is very important.

Run:

```
python manage.py test -v 2
```

The project already contains tests for things such as:

- cheapest fitting box
- weight limits
- quantity
- rotated products
- products that don't fit
- exact dimensions
- multiple products
- tie-breaking
- invalid input
- empty orders
- API errors

So first see whether the existing implementation passes.

Send me the complete output of:

```
python manage.py test -v 2
```

Then I'll tell you the next step.

Step 8 — Understand what the existing solution actually does

Before modifying it, you should understand the core logic because the assignment says your submission must clearly show your own thinking. Assignment - Python_Django Deve...

The important file is:

```
boxes/services.py
```

The basic flow is:

```
Order
  ↓
Get products + quantities
  ↓
Calculate total weight
  ↓
Calculate total volume
  ↓
Check each available box
  ↓
Check weight limit
  ↓
Check volume
  ↓
Actually try to pack the products
  ↓
Find boxes that work
  ↓
Choose cheapest box
  ↓
Return recommendation
```

For example:

```
Product:
10 × 10 × 12 cm
Weight: 0.4 kg

Small Box:
20 × 15 × 10 cm
Max weight: 5 kg
```

Even though the volume may fit, the product cannot fit in its original orientation because its height is 12 cm.

The code therefore tries rotations.

That's one of the stronger parts of this solution.

Step 9 — Test the API manually

Once the tests pass, start the server:

```
python manage.py runserver
```

You'll get something similar to:

```
Starting development server at http://127.0.0.1:8000/
```

There are two API endpoints in the project.

Endpoint 1

```
GET /api/orders/<order_id>/recommend-box/
```

Endpoint 2

```
POST /api/recommend-box/
```

The second one accepts JSON like:

```
{
    "items": [
        {
            "product_id": 1,
            "quantity": 2
        }
    ]
}
```

Step 10 — Create sample data

You need Products and Boxes in the database to test the API properly.

The easiest way is to use Django admin.

Create an admin user:

```
python manage.py createsuperuser
```

Enter:

```
Username:
Email:
Password:
```

Then open:

`http://127.0.0.1:8000/admin/`

Log in.

Create some **Boxes**.

For example:

Box	L	W	H	Max Weight	Cost
Small	20	15	10	5	1.00
Medium	40	30	20	15	2.50
Large	60	40	40	30	4.00

Then create some Products.

For example:

Product	L	W	H	Weight
Mug	10	10	12	0.4
Book	20	14	3	1
Weight	15	10	8	8

Step 11 — Test the API

Use Postman, Thunder Client, or curl.

For example:

`POST http://127.0.0.1:8000/api/recommend-box/`

Body:

```
{
    "items": [
        {
            "product_id": 1,
            "quantity": 1
```

```
        }
    ]
}
```

You should get a response containing something like:

```
{
    "total_weight": 0.4,
    "total_volume": 1200.0,
    "reason": "Cheapest box that fits the weight and dimensions.",
    "recommended_box": {
        "id": 1,
        "name": "Small",
        "cost": 1.0
    }
}
```

Step 12 — Check the existing README

Open:

`README.md`

We need to verify whether it properly explains:

- project purpose
- setup
- database
- API
- algorithm
- testing
- assumptions
- example request/response

Don't blindly submit the existing README.

Send me its contents or I can help you improve it after we verify the application.

Step 13 — Check `AI_USAGE.md`

This is particularly important because the assignment explicitly requires it.

The assignment says it must contain:

1. AI tools used
2. Prompts given
3. Output accepted
4. Output rejected/modified
5. Mistakes made by AI
6. How you verified the final code Assignment - Python_Django Deve...

The ZIP already contains:

`AI_USAGE.md`

But **you need to make sure it accurately represents your actual AI usage**.

Don't invent AI usage that didn't happen.

Step 14 — VERY IMPORTANT: Chat transcript

The assignment says:

> Export the chat transcript and attach and include in your repository.

And it specifically says **not to generate this using AI**. Assignment - Python_Django Deve...

So I **will not write that transcript for you**.

You should export/save your actual ChatGPT conversation yourself and put it in the repository.

For example:

`chat-transcript.pdf`

or whatever format the hiring company accepts.

Step 15 — "What did you learn?"

Again, the assignment explicitly says not to use AI to generate this answer. Assignment - Python_Django Deve...

So write this yourself based on what **you actually learned**.

I can help you understand the project, but you should write that section in your own words.

Step 16 — Add more test cases if necessary

The existing project already has a fairly good test suite.

But after we run it, we should check whether there are important missing cases.

For example:

```
1. One small product
2. Multiple products
3. Product requiring rotation
4. Product exceeding weight limit
5. Product exceeding dimensions
6. Multiple quantities
7. No suitable box
8. Exact-fit box
9. Invalid quantity
10. Invalid dimensions
11. Empty order
12. Same-cost boxes
```

The assignment specifically requires test cases and test output. Assignment - Python_Django Deve...

Step 17 — Generate actual TEST_OUTPUT.md

The ZIP already has:

`TEST_OUTPUT.md`

But don't just trust it.

After running:

```
python manage.py test -v 2
```

copy the **actual terminal output** into `TEST_OUTPUT.md`.

The assignment allows either GitHub Actions output/link or terminal output in `TEST_OUTPUT.md`.

Assignment - Python_Django Deve...

Since this project already has:

```
.github/workflows/tests.yml
```

we can also set up GitHub Actions.

Step 18 — Put it on GitHub

After everything works locally:

```
git init
git add .
git commit -m "Initial box selection system"
```

Then create a GitHub repository.

After that:

```
git remote add origin YOUR_REPOSITORY_URL
git branch -M main
git push -u origin main
```

Your `.gitignore` is already present, which is good.

Do not upload:

```
venv/
__pycache__/
db.sqlite3
.env
```

Step 19 — Verify GitHub Actions

Once pushed, go to:

```
GitHub repository
→ Actions
→ tests
```

The workflow will run:

```
pip install -r requirements.txt
python manage.py test -v 2
```

If it passes, you'll have a strong piece of evidence for the assignment.

Step 20 — Final submission checklist

Before submitting, your repository should approximately look like:

```
box-selection-system/
│
├── boxes/
│   ├── migrations/
│   ├── tests/
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── services.py
│   ├── urls.py
│   └── views.py
│
├── config/
│
├── .github/
│   └── workflows/
│       └── tests.yml
│
├── manage.py
├── requirements.txt
├── README.md
├── AI_USAGE.md
├── TEST_OUTPUT.md
├── chat-transcript.pdf
└── .gitignore
```

Then submit:

1. **GitHub repository link**
2. **README.md**
3. **AI_USAGE.md**
4. **Your actual exported chat transcript**
5. **Your own "What did you learn?" answer**
6. **Test cases**
7. **Test-run output / GitHub Actions link** Assignment - Python_Django Deve...

For now, do ONLY these 4 things

Don't jump ahead.

1.

Extract the ZIP.

2.

Open it in VS Code.

3.

Run:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

4.

Run:

```
python manage.py check
```

Then:

```
python manage.py test -v 2
```

Send me the output of those two commands.

I'll then take you through the **next exact step**, including checking whether the existing solution actually satisfies the assignment and what, if anything, we need to modify. ‹/›

You said:

i have not created a virtual environmnet will it create an issue

ChatGPT said:

No, **not creating a virtual environment will not damage the project**. You can technically run the Django project without one.

However, I **strongly recommend creating one for this assignment** because it keeps the project's Django/Python packages separate from your other projects.

Why use a virtual environment?

Without one:

```
Your computer
 ├── Django version A
 ├── other Python packages
 ├── packages from other projects
 └── box-selection-system
```

Packages from different projects can conflict.

With one:

```
Your computer
 ├── Python
 │
 └── box-selection-system
      └── venv
          ├── Django
          ├── dependencies
          └── project packages
```

So if something goes wrong, it won't affect your other projects.

For your assignment, do this

Since you haven't created one yet, that's completely fine. Open the project folder in VS Code and run:

```
python -m venv venv
```

Then:

```
venv\Scripts\activate
```

You should see:

```
(venv) PS C:\...\box-selection-system>
```

Then install the project's dependencies:

```
pip install -r requirements.txt
```

Then:

```
python manage.py check
```

And finally:

```
python manage.py test -v 2
```

Important: `venv` should not be uploaded to GitHub. The project's `.gitignore` should contain `venv/`.

So you're actually at the **right point to create it now**. No need to restart or change anything in the project.

You said:

next what should i do should i create a new repository on github

ChatGPT said:

Yes — **you should create a new GitHub repository for this assignment**, but **do it after you confirm the project works locally**.

The assignment explicitly requires a **GitHub repository link** as part of the submission.

Assignment - Python_Django Deve...

Your order should be:

1. Finish local setup first

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

2. Check the project

```
python manage.py check
```

3. Run tests

```
python manage.py test -v 2
```

If those work, **then create the GitHub repository**.

When creating the GitHub repo

Go to GitHub → **New repository**.

I suggest a name like:

`box-selection-system`

or:

`AI-Assisted-Box-Selection-System`

Keep it **Public** if the assignment doesn't specify otherwise, because they need to access your GitHub repository.

When creating it, **do NOT add**:

- README
- `.gitignore`
- License

because your project already has these files.

Then we'll connect your local project to that repository.

Important

Don't upload the `venv` folder. Your `.gitignore` should exclude it.

So right now, don't create the GitHub repo yet.

First run:

```
python manage.py check
```

and

```
python manage.py test -v 2
```

Send me the output. **After we confirm everything works, I'll give you the exact Git commands to create the GitHub repo and push this project.**

You said:

```
(venv) PS C:\Users\Admin\Desktop\Mayuri_Barkade_Test-Solution\box-selection-system> python manage.py test -v 2
Found 25 test(s).
Creating test database for alias 'default' ('file:memorydb_default?mode=memory&cache=shared')...
Operations to perform:
  Synchronize unmigrated apps: boxes, messages, staticfiles
  Apply all migrations: admin, auth, contenttypes, sessions
Synchronizing apps without migrations:
  Creating tables...
    Creating table boxes_product
    Creating table boxes_box
    Creating table boxes_order
    Creating table boxes_orderitem
    Running deferred SQL...
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
System check identified no issues (0 silenced).
test_bad_json_400 (boxes.tests.test_api.RecommendApiTests.test_bad_json_400) ... ok
test_empty_order_400 (boxes.tests.test_api.RecommendApiTests.test_empty_order_400) ... ok
test_get_not_allowed_on_adhoc
```

(boxes.tests.test_api.RecommendApiTests.test_get_not_allowed_on_adhoc) ... ok
test_heavy_order_gets_medium
(boxes.tests.test_api.RecommendApiTests.test_heavy_order_gets_medium) ... ok
test_nothing_fits_returns_422
(boxes.tests.test_api.RecommendApiTests.test_nothing_fits_returns_422) ... ok
test_order_endpoint_recommends_cheapest_box
(boxes.tests.test_api.RecommendApiTests.test_order_endpoint_recommends_cheapest_box) ... ok
test_unknown_order_404
(boxes.tests.test_api.RecommendApiTests.test_unknown_order_404) ... ok
test_unknown_product_404
(boxes.tests.test_api.RecommendApiTests.test_unknown_product_404) ... ok
test_cheaper_bigger_box_wins_over_pricier_smaller_box
(boxes.tests.test_services.RecommendBoxTests.test_cheaper_bigger_box_wins_over_pricier_smaller_box) ... ok
test_empty_order_raises
(boxes.tests.test_services.RecommendBoxTests.test_empty_order_raises) ... ok
test_exact_fit_is_accepted
(boxes.tests.test_services.RecommendBoxTests.test_exact_fit_is_accepted) ... ok
test_input_boxes_not_mutated_and_result_deterministic
(boxes.tests.test_services.RecommendBoxTests.test_input_boxes_not_mutated_and_result_deterministic) ... ok
test_item_too_long_for_every_box
(boxes.tests.test_services.RecommendBoxTests.test_item_too_long_for_every_box) ... ok
test_multiple_items_pack_together
(boxes.tests.test_services.RecommendBoxTests.test_multiple_items_pack_together) ... ok
test_negative_weight_raises
(boxes.tests.test_services.RecommendBoxTests.test_negative_weight_raises) ... ok
test_no_boxes_defined
(boxes.tests.test_services.RecommendBoxTests.test_no_boxes_defined) ... ok
test_non_positive_dimension_raises

```
(boxes.tests.test_services.RecommendBoxTests.test_non_positive_dimension_raises) ... ok
test_quantity_adds_weight_and_volume
(boxes.tests.test_services.RecommendBoxTests.test_quantity_adds_weight_and_volume) ... ok
test_rotation_is_allowed
(boxes.tests.test_services.RecommendBoxTests.test_rotation_is_allowed) ... ok
test_single_small_item_picks_cheapest_fitting
(boxes.tests.test_services.RecommendBoxTests.test_single_small_item_picks_cheapest_fitting) ... ok
test_tie_on_cost_prefers_smaller_volume
(boxes.tests.test_services.RecommendBoxTests.test_tie_on_cost_prefers_smaller_volume) ... ok
test_too_heavy_for_every_box
(boxes.tests.test_services.RecommendBoxTests.test_too_heavy_for_every_box) ... ok
test_volume_ok_but_shape_wrong
(boxes.tests.test_services.RecommendBoxTests.test_volume_ok_but_shape_wrong) ... ok
test_weight_forces_bigger_box
(boxes.tests.test_services.RecommendBoxTests.test_weight_forces_bigger_box) ... ok
test_zero_quantity_raises
(boxes.tests.test_services.RecommendBoxTests.test_zero_quantity_raises) ... ok

----------------------------------------------------------------------
Ran 25 tests in 0.170s

OK
Destroying test database for alias 'default' ('file:memorydb_default?mode=memory&cache=shared')...
```

ChatGPT said:

Perfect. **Your project is passing all tests.** ✅

The important part of your output is:

```
System check identified no issues (0 silenced).

Ran 25 tests in 0.170s
```

```
OK
```

So currently:

- ✅ Django system check passes
- ✅ **25/25 tests pass**
- ✅ No test failures
- ✅ API tests pass
- ✅ Box-selection logic tests pass
- ✅ Rotation logic is tested
- ✅ Weight/dimension validation is tested
- ✅ Multiple-item packing is tested
- ✅ Error cases are tested

This is a very good starting point.

Now do this next: create the GitHub repository

Since the local project is working, **yes, now create the new GitHub repository.**

1. Go to GitHub

Click **New repository**.

Use:

Repository name:

```
box-selection-system
```

For visibility, I recommend:

```
Public
```

2. IMPORTANT: Don't initialize it

On the GitHub creation page, leave these unchecked:

- ❌ Add a README file
- ❌ Add .gitignore
- ❌ Choose a license

Because your project already contains these files.

---

3. After creating the repository

GitHub will show you a URL similar to:

`https://github.com/YourUsername/box-selection-system.git`

Don't run the commands from GitHub blindly yet.

Come back here and send me the **repository URL**.

Then I'll give you the exact commands for your current folder:

`C:\Users\Admin\Desktop\Mayuri_Barkade_Test-Solution\box-selection-system`

We'll do it step-by-step:

```
Local project
     ↓
Git init
     ↓
Check .gitignore
     ↓
git add
     ↓
git commit
     ↓
Connect GitHub
     ↓
git push
     ↓
Check GitHub
     ↓
README / AI_USAGE / TEST_OUTPUT review
     ↓
GitHub Actions
     ↓
Final submission
```

Don't modify any code right now. Your 25/25 test result means the implementation is currently in a good state.

You said:

https://github.com/Mayuri-Barkade/box-selection-system

this is my github respotory link

ChatGPT said:

Perfect. I checked the repository, and **`box-selection-system` is currently empty** on GitHub, so we haven't uploaded anything yet. GitHub

That's actually good — we can push your working local project cleanly.

Next step: connect your local project to GitHub

You are currently in:

`C:\Users\Admin\Desktop\Mayuri_Barkade_Test-Solution\box-selection-system`

and your virtual environment is active.

Step 1 — Check Git status

Run:

```
git status
```

If it says something like:

```
fatal: not a git repository
```

that's completely fine.

Step 2 — Initialize Git

Run:

```
git init
```

Step 3 — Check your `.gitignore`

Before adding anything, run:

```
type .gitignore
```

Make sure it includes at least:

```
venv/
__pycache__/
*.pyc
```

```
db.sqlite3
.env
```

Do not add the `venv` folder to GitHub.

Step 4 — Add the project

Run:

```
git add .
```

Then:

```
git status
```

Look carefully at the files Git is going to upload.

You should **NOT** see things like:

```
venv/
__pycache__/
*.pyc
```

Step 5 — Commit

Run:

```
git commit -m "Initial commit - box selection system"
```

Step 6 — Connect your GitHub repository

Run:

```
git remote add origin https://github.com/Mayuri-Barkade/box-selection-system.git
```

Then verify:

```
git remote -v
```

You should see your GitHub repository URL.

Step 7 — Push

Run:

```
git branch -M main
```

Then:

```
git push -u origin main
```

⚠️ One important thing

Don't run all of these blindly at once.

Start with just:

```
git status
```

Send me the output.

I'll check it and then tell you exactly what to run next. This way we avoid accidentally uploading `venv`, cache files, database files, or anything else that shouldn't be in the repository.

You said:

done

ChatGPT said:

Great 👍

Now continue with the Git setup.

Step 1 — Initialize Git

Run:

```
git init
```

Step 2 — Check your `.gitignore`

Run:

```
type .gitignore
```

You should see entries that exclude at least:

```
venv/
__pycache__/
*.pyc
db.sqlite3
.env
```

Step 3 — Add the files

If the `.gitignore` looks correct:

```
git add .
```

Then check:

```
git status
```

Send me the output of `git status` before committing.

I'll check that you're not accidentally about to upload `venv`, cache files, or other unnecessary files.

You said:

On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

ChatGPT said:

Good — that means your local Git repository is already connected to GitHub and currently has **nothing new to commit**.

The important line is:

```
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
```

So **don't run `git add` or `git commit` again yet.**

Next, let's verify what's actually on GitHub

Run these two commands:

```
git remote -v
```

and:

```
git log --oneline --max-count=5
```

Send me the output of both.

I want to confirm that your **box-selection-system files are actually committed and connected to the correct GitHub repository** before we move on to the next part of the assignment.

You said:

```
(venv) PS C:\Users\Admin\Desktop\Mayuri_Barkade_Test-Solution\box-selection-system> git remote -v
origin  https://github.com/Mayuri-Barkade/box-selection-system.git (fetch)
origin  https://github.com/Mayuri-Barkade/box-selection-system.git (push)
(venv) PS C:\Users\Admin\Desktop\Mayuri_Barkade_Test-Solution\box-selection-system> git log --oneline --max-count=5
ae4ae2e (HEAD -> main, origin/main) Initial commit - box selection system
```

ChatGPT said:

Perfect. ✅ Everything is connected correctly.

Your output confirms:

`origin → https://github.com/Mayuri-Barkade/box-selection-system.git`

and:

`ae4ae2e (HEAD -> main, origin/main) Initial commit - box selection system`

So:

- ✅ Local project is a Git repository
- ✅ GitHub remote is correct
- ✅ You're on `main`
- ✅ Your local `main` matches GitHub `main`
- ✅ Initial commit exists
- ✅ Project has been pushed successfully

Now let's move to the assignment itself.

The next thing I recommend is **not adding more code yet**.

We need to check what you've actually submitted against the assignment requirements:

Requirement	Current status
Django box-selection system	✅ Working
25 automated tests	✅ 25/25 passing

Requirement	Current status
GitHub repository	✅ Done
README.md	🔍 Need to review
AI_USAGE.md	🔍 Need to review
Chat transcript	⏳ You must export your actual transcript yourself
"What did you learn?"	⏳ You must write this yourself
Test cases	✅ Tests exist; need to document them
Test output	🔍 Need to generate/update
GitHub Actions	🔍 Need to verify

The assignment specifically requires all of these items. Assignment - Python_Django Deve...

So our next step is:

Let's inspect `README.md`, `AI_USAGE.md`, and `TEST_OUTPUT.md` before changing anything.

Since those files are already in your project, open them in VS Code and **paste their contents here** (you can send them one at a time).

Start with `README.md`.

I'll check it against the assignment and tell you exactly what needs to be added/changed.

You said:

AI-Assisted Box Selection System (Django)

Recommends the most suitable shipping box for an order, based on product
dimensions/weight and each box's internal dimensions, max weight and cost.

Units: dimensions in **cm**, weight in **kg**.

How it works

The logic lives in `boxes/services.py` (plain Python, no Django imports, so it
is easy to test). For each box it checks:

1. **Weight** - total order weight <= box max weight.
2. **Volume** - total item volume <= box volume (cheap early rejection).
3. **Packing** - items must physically fit. A guillotine packer tries all 6 rotations of each item (largest first) with bounded backtracking.

Of the boxes that pass, the **cheapest** wins. Ties go to the smaller volume,
then lower max weight, then name, so the answer is deterministic.

Design decisions / limitations

- The packer is a heuristic. If it says "fits", the items really fit (no overlaps). If it says "doesn't fit", a perfect packer *might* still find a way. I chose this direction because shipping a too-small box is worse than picking a slightly larger one.
- Search is capped (`MAX_SEARCH_NODES`) so huge orders can't hang the request; hitting the cap counts as "doesn't fit".
- Items are treated as rigid rectangular blocks. No fragility, "this side up", or stacking-weight rules.
- Box *internal* dimensions are assumed to be the usable space.
- The ad-hoc POST endpoint is CSRF-exempt and unauthenticated. Add auth before real use.

Setup

```bash
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python manage.py migrate
```

```
python manage.py createsuperuser   # to use /admin/ for products, boxes, orders
python manage.py runserver
```

Add Products, Boxes and Orders (with items) in `/admin/`.

API

`GET /api/orders/<order_id>/recommend-box/`

`POST /api/recommend-box/`
```json
{"items": [{"product_id": 1, "quantity": 2}]}
```

Example success (200):
```json
{
  "recommended_box": {"id": 1, "name": "Small", "cost": 1.0},
  "total_weight": 0.8,
  "total_volume": 2400.0,
  "reason": "Cheapest box that fits the weight and dimensions.",
  "rejected_boxes": []
}
```

Status	Meaning
200	A box was found
400	Bad input (empty order, bad JSON, invalid quantity/size)
404	Order or product not found
422	Valid order, but no box can hold it (`recommended_box` is null, `rejected_boxes` says why)

Tests

```bash
python manage.py test -v 2
```

- `boxes/tests/test_services.py` - selection logic (weight limits, rotation, exact fit, ties, quantities, invalid input, determinism).
- `boxes/tests/test_api.py` - endpoints and status codes.

CI: `.github/workflows/tests.yml` runs the suite on every push.
Test output: see `TEST_OUTPUT.md`.

ChatGPT said:

Your README is already **quite strong**. It explains the algorithm, design decisions, setup, API, limitations, tests, and CI. I would **not rewrite it completely**.

However, before submission, I would make a few important improvements.

What is already good

- ✅ Clearly explains the purpose
- ✅ Explains the selection algorithm
- ✅ Mentions all 6 rotations
- ✅ Explains the cheapest-box selection
- ✅ Includes limitations/assumptions
- ✅ Gives setup instructions
- ✅ Documents both API endpoints
- ✅ Shows an example request/response
- ✅ Documents HTTP status codes
- ✅ Explains the test files
- ✅ Mentions GitHub Actions

That matches the technical nature of the assignment well. The assignment asks you to demonstrate your own thinking, and your **Design decisions / limitations** section is especially useful for that.

Assignment - Python_Django Deve...

But I would change these 4 things

1. Fix the formatting

Your pasted README contains things like:

```
**# AI-Assisted Box Selection System (Django)**
```

and:

\`boxes/services.py\`

If those backslashes and extra ** are actually present in the file, the Markdown formatting will look wrong on GitHub.

It should be:

AI-Assisted Box Selection System (Django)

and:

`boxes/services.py`

Check this in the actual GitHub-rendered README.

---

2. Add the project structure

This will make it easier for the reviewer to understand your project.

For example:

Project Structure

```
box-selection-system/
├── boxes/
│   ├── models.py
│   ├── services.py
│   ├── views.py
│   ├── urls.py
│   └── tests/
├── config/
├── .github/
│   └── workflows/
├── manage.py
├── requirements.txt
├── README.md
├── AI_USAGE.md
└── TEST_OUTPUT.md
```

---

3. Add an "Approach" section

Your current **How it works** is good, but I'd make the actual decision process slightly clearer:

```
## Selection Approach

For each candidate box:

1. Calculate the total order weight.
2. Reject the box if the weight exceeds its capacity.
3. Calculate total item volume.
4. Reject the box if the total volume exceeds the box volume.
5. Run the packing algorithm to verify that the individual products can physically
fit.
6. Keep all boxes that successfully pass these checks.
7. Select the cheapest valid box.
8. Apply deterministic tie-breaking when multiple boxes have the same cost.
```

This makes your logic very easy for an interviewer/reviewer to follow.

---

4. Add a clear "Test Results" section

Since you have **25/25 tests passing**, that's worth showing.

For example:

```
## Test Results

The project currently has 25 automated tests covering:

- Box selection
- Weight constraints
- Dimension constraints
- Product rotation
- Multiple products
- Product quantities
- Exact-fit cases
- Tie-breaking
- Invalid input
- API error handling

Latest test run:
```

```
Ran 25 tests in 0.170s

OK
```

Your actual terminal output confirms all 25 passed.

One thing I DON'T want you to change

Keep this section:

> The packer is a heuristic...

That's actually a **good engineering explanation**.

If an interviewer asks:

"Does your algorithm guarantee the optimal packing?"

you can honestly say:

> "No. It uses bounded backtracking with a heuristic. If it confirms that an order fits, the placement is valid, but failing to find a placement doesn't mathematically prove that no possible arrangement exists. I chose this approach to keep the API predictable and bounded."

That's much better than pretending the algorithm is perfect.

What we should do next

Don't edit the README yet.

Next, send me the contents of:

`AI_USAGE.md`

This one is **more important than README**, because the assignment has very specific requirements for it:

- AI tools used
- prompts
- accepted output
- rejected/modified output

- AI mistakes
- how you verified the code Assignment - Python_Django Deve...

We'll check that next, **without inventing anything that you didn't actually do**.

You said:

do we need to create something visual in this assignment

ChatGPT said:

No. Based on the assignment document, **there is no requirement to create a visual UI, diagram, flowchart, or frontend**.

The assignment asks you to build a **Django-based system** that recommends the most suitable shipping box for an order. Assignment - Python_Django Deve...

What they actually require

The submission checklist is:

1. **GitHub repository link**
2. **README.md**
3. **AI_USAGE.md**
4. **Exported chat transcript**
5. **"What did you learn?"** response
6. **Test cases**
7. **Test run output / GitHub Actions link** Assignment - Python_Django Deve...

There is **no mention of**:

- ❌ Frontend
- ❌ HTML/CSS page
- ❌ Dashboard
- ❌ UI screenshots
- ❌ Flowchart
- ❌ System architecture diagram
- ❌ Database ER diagram
- ❌ Box visualization

So your current approach is correct

Your project can be essentially:

```
Django
   ↓
Products + Boxes + Orders
   ↓
Box-selection algorithm
   ↓
REST API
   ↓
Recommended box
```

The important part is that the **box-selection logic works correctly**, and your current test run is excellent:

```
25 tests
25 passed
0 failed
```

One thing that could be useful, but is NOT required

You could include a **simple API example** in the README, which you already have:

```
{
  "items": [
    {
      "product_id": 1,
      "quantity": 2
    }
  ]
}
```

That's actually more relevant to this assignment than creating a visual interface.

So don't spend time building a frontend just to make it look visual. Focus now on making the required documentation, tests, AI usage record, and GitHub submission solid.

You said:

AI Usage

> TO FILL IN YOURSELF, honestly, from what you actually did. Delete these
> hints when done. The employer wants to see your own thinking, so use your own
> words and your real prompts.

Which AI tool(s) I used
<!-- e.g. tool name + what you used it for -->

The prompts I gave
<!-- paste the real prompts you sent, in order -->

What output I accepted
<!-- which parts you kept as-is, and why -->

What output I rejected or modified
<!-- what you changed and why -->

Mistakes the AI made
<!-- bugs, wrong assumptions, things you caught -->

How I verified the final code
<!-- tests you ran, manual checks, edge cases you tried -->

ChatGPT said:

Yes — this `AI_USAGE.md` is intentionally a **template**, and the assignment wants you to fill it with your **actual AI usage**, not a generic answer. The assignment explicitly asks for the tools, real prompts, accepted/rejected output, AI mistakes, and verification. Assignment - Python_Django Deve...

Because the assignment says to describe **what you actually did**, I don't want to invent prompts or claim that you used AI in ways you didn't.

From **this conversation**, we can accurately document that you used ChatGPT for things like:

- Understanding the assignment requirements
- Deciding the next steps for setting up the project
- Understanding virtual environments
- Git/GitHub setup
- Reviewing the README
- Reviewing whether a visual UI was required
- Reviewing the test results

But **I cannot truthfully fill in the "prompts I gave" section with prompts from before this conversation**, because I don't have evidence of what you used when the original solution was created.

What I recommend

Keep the existing `AI_USAGE.md` template for now.

You should personally fill in:

Which AI tools I used
→ e.g. ChatGPT, Claude, Cursor, etc., whichever you actually used.

The prompts I gave
→ Copy your actual prompts from your ChatGPT/Claude/Cursor history.

What output I accepted
→ Explain which AI-generated suggestions you actually used.

What output I rejected or modified
→ Mention things you changed instead of blindly accepting.

Mistakes the AI made
→ If you caught any incorrect suggestions, document them. If there weren't any, say so honestly.

How I verified the final code
→ This part we now know from your actual testing:

```
python manage.py check
python manage.py test -v 2
```

Your test result was:

```
System check identified no issues (0 silenced).

Ran 25 tests in 0.170s

OK
```

So you can definitely document that verification.

One important point

Don't write something like:

> "ChatGPT generated the complete project and I accepted it."

unless that's actually what happened.

The employer is specifically asking to see **your own thinking**, so an honest record of what you used AI for is much safer and more credible.

For now, leave `AI_USAGE.md` unchanged. We can come back to it after we've finished checking the rest of the project, and I'll help you organize the information you've actually provided without fabricating anything.

You said:

so whats next

ChatGPT said:

Now that the **project works locally**, **25/25 tests pass**, and the GitHub repository is connected, we should move through the assignment checklist systematically.

Next: verify GitHub Actions

Your README says:

> `.github/workflows/tests.yml` runs the suite on every push.

So let's confirm that it actually works on GitHub.

1. Make a tiny harmless change

For example, open `README.md` and add one line at the bottom:

```
## Status

All automated tests currently pass.
```

Save it.

2. Commit the change

```
git add README.md
git commit -m "Update README with test status"
```

3. Push it

```
git push
```

4. Go to your GitHub repository

Your box-selection-system repository

Then:

Actions → Tests

You should see a workflow run triggered by your push.

What we're checking

We want to see:

```
GitHub Actions
     ↓
Install dependencies
     ↓
Run Django tests
     ↓
25 tests
     ↓
✓ Passed
```

If the workflow shows a **green check**, we're done with CI.

After that, our order will be:

1. ✅ Local tests — done
2. 🔄 GitHub Actions — do now
3. ⏭️ Review `TEST_OUTPUT.md`
4. ⏭️ Review/finalize `README.md`
5. ⏭️ Fill `AI_USAGE.md` honestly
6. ⏭️ Add/document test cases
7. ⏭️ You export your actual chat transcript yourself
8. ⏭️ You write your own "What did you learn?" section
9. ⏭️ Final GitHub repository check
10. ⏭️ Submit the repository link

Do the README change + `git push` now, then tell me what you see under GitHub → Actions.